



Data Re-uploading for a Universal Quantum Classifier

Achieving Universality with a Single Qubit

**GROUP 2612
ELENA-EYLUL-MELISSA-SAHASRA**

INTRODUCTION and MOTIVATION

AIM of the paper: Finding absolute minimum of quantum resources for general supervised classification.

Standard Approach

Uses complex encoding circuits to introduce non-linearities.

DRAWBACK:

Requires many qubits, increasing spatial complexity.

Proposed Approach

- **Seamless Integration:** No division between "encoding" and "processing".
- **Data Re-uploading:** Iteratively re-introducing classical data.
- **Strategic Trade-off:** Exchanging spatial complexity for depth complexity.

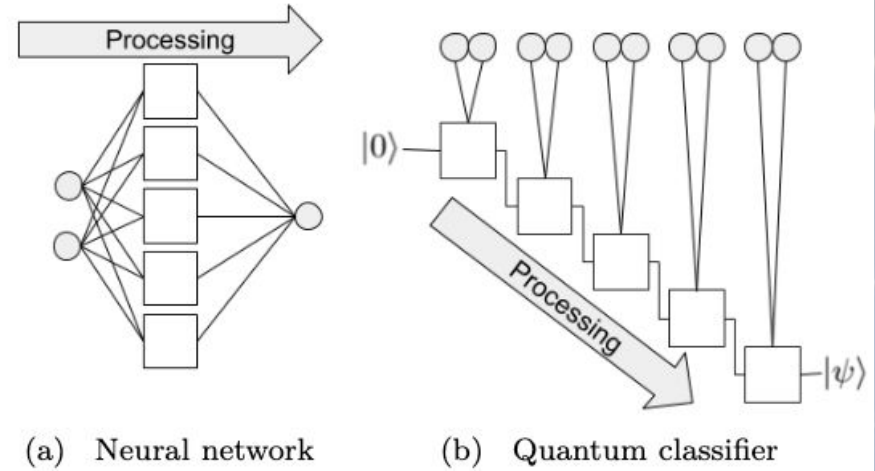
CLASSICAL **vs.** QUANTUM

Neural Network (Spatial)

Every neuron in a hidden layer receives the input data simultaneously through spatial distribution.

Single Qubit Classifier (Sequential)

Receives info from previous units and classically re-introduced input data in a sequential sequence.



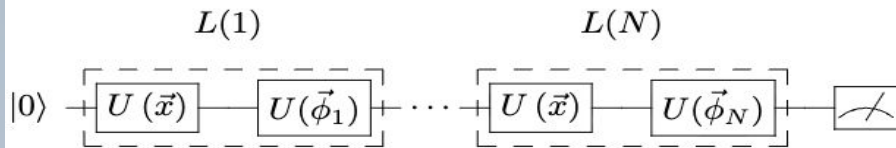
"The quantum circuit mimics a hidden layer by folding the network structurally into a sequence of operations."

THE CIRCUIT SCHEME

(a) Original Scheme

Separates data uploading $U(\vec{x})$ and processing parameters $U(\vec{\phi})$.

Each layer $L(i)$ requires multiple sequential gates.



(a) Original scheme

(b) Compressed Scheme

Fuses data and tunable parameters into a single unitary gate $U(\phi, x)$.

Halves the circuit depth while maintaining exact representational power.



(b) Compressed scheme

"Compression maintains the exact same representational power while optimizing circuit depth."

STRUCTURE OF THE CLASSIFIER

Processing Unit (Layer)

The mathematical foundation of the single-qubit processing gate.

$$L(i) = U \left(\vec{\theta}_i + \vec{w}_i \circ \vec{x} \right)$$

Operation

Hadamard product of weights and input.

$$\vec{w}_i \circ \vec{x} = (w_i^1 x^1, w_i^2 x^2, w_i^3 x^3)$$

Data Vectors (3D) and Biases

$\vec{x}$: Classical data is partitioned into k vectors of size 3. Padding is applied if dimensions are < 3.

$\vec{\theta}_i$: Trainable biases are applied per layer.

$$L(i) = U \left(\vec{\theta}_i^{(k)} + \vec{w}_i^{(k)} \circ \vec{x}^{(k)} \right) \cdots U \left(\vec{\theta}_i^{(1)} + \vec{w}_i^{(1)} \circ \vec{x}^{(1)} \right)$$

Sequential Layer Expansion

"Complexity of circuit increases linearly with the size of the input space."

THE ORIGIN OF NON-LINEARITY

The Paradox: Data points "x" are introduced linearly into the argument of the rotational gate.

$$(\vec{\theta}_i + \vec{w}_i \circ \vec{x})$$

The Solution

- Non-linearity comes strictly from the intrinsic structure of the SU(2) rotational gates.
- SU(2) matrices naturally consist of trigonometric operations.

"The inherent trigonometric nature of quantum gates enables complex non-linear mapping through layer stacking."

Key Concept

While a single rotation has limited representation, applying **"subsequent rotations (layers)"**

Compounding effect generates highly non-trivial, non-linear functions of the original data.

$$U(\vec{\phi}) = e^{i\vec{\omega}(\vec{\phi}) \cdot \vec{\sigma}},$$

with $\vec{\omega}(\vec{\phi}) = (\omega_1(\vec{\phi}), \omega_2(\vec{\phi}), \omega_3(\vec{\phi}))$ and

$$\omega_1(\vec{\phi}) = d \mathcal{N} \sin((\phi_2 - \phi_3)/2) \sin(\phi_1/2),$$

$$\omega_2(\vec{\phi}) = d \mathcal{N} \cos((\phi_2 - \phi_3)/2) \sin(\phi_1/2),$$

$$\omega_3(\vec{\phi}) = d \mathcal{N} \sin((\phi_2 + \phi_3)/2) \cos(\phi_1/2),$$

MEASUREMENTS

Encoding + Processing



Measurement of
Resulting State



Cost Function
Computation

How to associate outputs to label classes?

The Easy Way

Assigning to the classes a label state. For 2 classes (A, B): if $P(0) > P(1)$, classify as A; otherwise, as B.

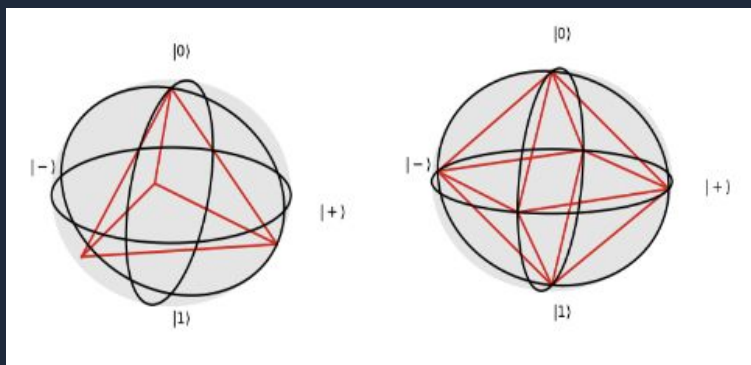
Generalization to N Classes

$$0 < \lambda_1 < \dots < \lambda_{n-1} < 1$$

$P(0)$ falls into one interval, and classification is issued accordingly.

MEASUREMENTS

Geometrical approach: For **N classes**, we identify the N maximally orthogonal points on the **Bloch sphere**.



4 classes

6 classes

Core Mechanism

Data point classification is achieved through precise rotation across the Bloch sphere's surface.

More Layers → **More Rotations** → **Feature Map** → **Class Separation**

COST FUNCTIONS

Fidelity Cost

$$\chi_f^2(\vec{\theta}, \vec{w}) = \sum_{\mu=1}^M \left(1 - |\langle \tilde{\psi}_s | \psi(\vec{\theta}, \vec{w}, \vec{x}_\mu) \rangle|^2 \right)$$

Bra: true label state

Ket: final state of the circuit

- Forces parameters to reach the maximum in fidelities.

Weighted Fidelity

$$\chi_{wf}^2(\vec{\alpha}, \vec{\theta}, \vec{w}) = \frac{1}{2} \sum_{\mu=1}^M \left(\sum_{c=1}^C \left(\alpha_c F_c(\vec{\theta}, \vec{w}, \vec{x}_\mu) - Y_c(\vec{x}_\mu) \right)^2 \right)$$

Calculated over all classes.

Expected fidelity of a successful classification.

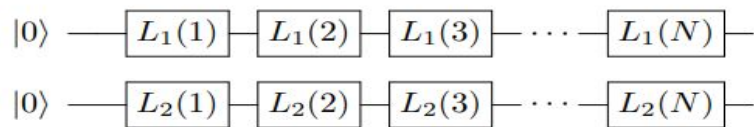
- Forces parameters to be close to a specified configuration.

MULTI-QUBIT QUANTUM CLASSIFIER

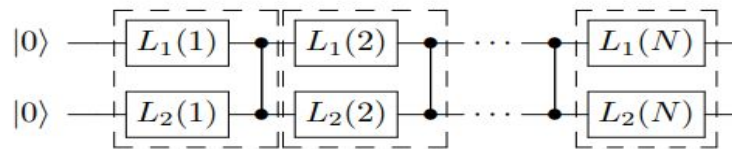
We can generalize the single-qubit results for multi-qubit systems to enhance computational capabilities.

Performance & Depth

- Introducing entanglement allows for significant performance increases.
- **Non-entangled:** Circuit depth scales as **N** for N layers.
- **Entangled:** Circuit depth scales as **2N**, doubling complexity.



(a) Ansatz with no entanglement



(b) Ansatz with entanglement

MULTI-QUBIT MEASUREMENTS

Strategy One

Natural generalization of the single-qubit approach, comparing the final circuit state with a target state.

Uses the same fidelity cost function, adapted for multiple qubits.

$$\chi_f^2(\vec{\theta}, \vec{w}) = \sum_{\mu=1}^M \left(1 - |\langle \tilde{\psi}_s | \psi(\vec{\theta}, \vec{w}, \vec{x}_\mu) \rangle|^2 \right)$$

Strategy Two

Focusing on a single qubit to associate classes based on its state via reduction.

$$\chi_{wf}^2(\vec{\alpha}, \vec{\theta}, \vec{w}) = \frac{1}{2} \sum_{\mu=1}^M \sum_{c=1}^C \left(\sum_{q=1}^Q \left(\alpha_{c,q} F_{c,q}(\vec{\theta}, \vec{w}, \vec{x}_\mu) - Y_c(\vec{x}_\mu) \right)^2 \right)$$

where:

$$F_{c,q}(\vec{\theta}, \vec{w}, \vec{x}) = \langle \tilde{\psi}_c | \rho_q(\vec{\theta}, \vec{w}, \vec{x}) | \tilde{\psi}_c \rangle,$$

ρ_q is the reduced density matrix of the qubit under consideration.

UNIVERSITY OF THE SINGLE QUBIT CLASSIFIER

Universal Approximation Theorem (UAT)

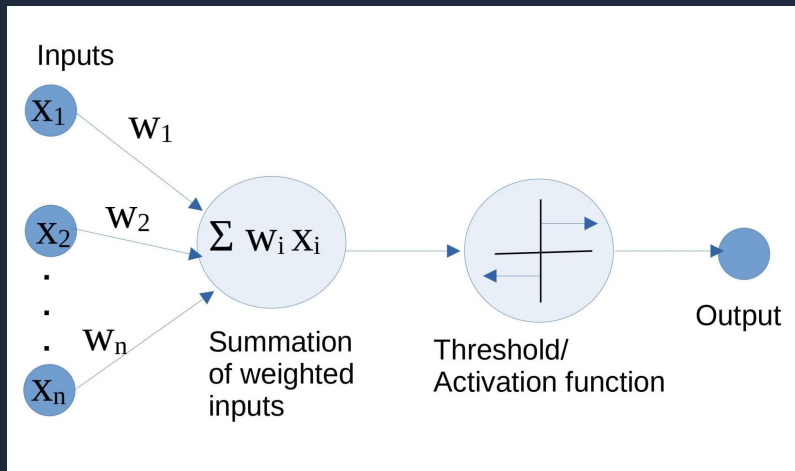
States that a feedforward neural network with a single hidden layer, containing a finite number of neurons (N) and suitable non-linear activation function, can approximate any continuous function (f) on a compact set to arbitrary accuracy.

$$h(\vec{x}) = \sum_{i=1}^N \alpha_i \varphi(\vec{w}_i \cdot \vec{x} + b_i)$$

Where h is an approximate realization of f with precision ϵ :

$$|h(\vec{x}) - f(\vec{x})| < \epsilon$$

Classical Neuron Model



UNIVERSAL QUANTUM CIRCUIT APPROXIMATION

It's possible to state something equivalent for a quantum circuit:

Universal Quantum Circuit Approximation:

Given the re-uploading process, any continuous function can be approximated by adding more layers that alter the initial input in a quantum circuit.

The final given state is a product between matrices. The clear correlation with the classical UAT occurs via the **BCH equation**.

Mapping Between Classical and Quantum Parameters		
Aspect	Neural Network	Quantum Circuit
Output Weights	α_i (neuron output weights)	α_c (class weights)
Neuron Weights	$\vec{w}_i$ (neuron weights)	$\vec{w}_i$ (input weights)
Bias Terms	b_i (bias of each neuron)	$\vec{\theta}_i$ (tunable angles)
Hidden Layer Size	N (number of neurons)	N (number of layers)
Activation Function	ϕ (nonlinear activation)	$\vec{\omega}$ (trigonometric functions)

This mapping highlights the direct structural analogy between classical neurons and multi-layered quantum gate rotations.

SU(2) ROTATIONAL MATRICES & DATA UPLOADING

Starting from this notation of the **SU(2) rotational matrices**: $U(\vec{\phi}) = e^{i\vec{\omega}(\vec{\phi}) \cdot \vec{\sigma}}$

Being σ the conventional Pauli matrices and ω the trigonometric functions, dependent on ϕ

$$\vec{\omega}(\vec{\phi}) = (\omega_1(\vec{\phi}), \omega_2(\vec{\phi}), \omega_3(\vec{\phi}))$$

The **single-qubit classifier** codifies the data points into ϕ parameters of the U unitary gate. If re-uploading data together with the tunable parameters:

$$\vec{\phi}(\vec{x}) = (\phi_1(\vec{x}), \phi_2(\vec{x}), \phi_3(\vec{x})) = \vec{\theta} + \vec{w} \circ \vec{x}.$$

It results in a **multiplication of the matrices**:

$$U(\vec{x}) = U_N(\vec{x})U_{N-1}(\vec{x}) \cdots U_1(\vec{x}) = \prod_{i=1}^N e^{i\vec{\omega}(\vec{\phi}_i(\vec{x})) \cdot \vec{\sigma}},$$

BAKER-CAMPBELL-HAUSDORFF SIMPLIFICATION

The serial structure can be simplified into a **sum of terms** applying manipulation with the Baker Campbell Hausdorff formula, resulting in:

$$\mathcal{U}(\vec{x}) = \exp \left[i \sum_{i=1}^N \vec{\omega}(\vec{\phi}_i(\vec{x})) \cdot \vec{\sigma} + \mathcal{O}_{corr} \right].$$

The BCH gives a expression for the logarithm of the product of two exponentials of non-commuting operators. In the case for Pauli matrices, the commutation term is:

$$\frac{1}{2}[i\alpha\sigma_i, i\beta\sigma_j] = \frac{1}{2}(-\alpha\beta)[\sigma_i, \sigma_j] = -i\alpha\beta\epsilon_{ijk}\sigma_k$$

So the remaining terms **Ocorr** are also proportional to Pauli matrices due to $[\sigma_i, \sigma_j] = 2i\epsilon_{ijk}\sigma_k$

To deal with these terms, it's possible to rewrite $\mathcal{U}(\mathbf{x})$ (SU(2) group composition law) as: and **Ocorr** gets absorbed into $\mathbf{f}(\mathbf{x})$

QUANTUM FUNCTION APPROXIMATION

$$\mathcal{U}(\vec{x}) = e^{i\vec{\xi}(\vec{x}) \cdot \vec{\sigma}} = e^{i\vec{f}(\vec{x}) \cdot \vec{\sigma} + i\vec{\varrho}(\vec{x}) \cdot \vec{\sigma}}$$

For each data point x there's a final state dependent on a $\xi(x)$ trigonometric function.

The weighted fidelity cost function can be constructed using the α new parameters for each class.

This function, a resulting combination of $\xi(x)$ and α is expected to be complex enough to probably represent almost any continuous function.

While in a classic Neural Network it's necessary to have enough N neurons, in a quantum circuit it takes N layers for the "equivalent" approximation!

MINIMIZATION METHODS

Training of single or multi-qubit quantum classifier: Minimization in parameter space.

What is the parameter Space of the circuit?

Parameters:

- Circuit angles $\{\theta_i\}$ in rotation gates
- Classical weights $\{w_i\}$ that map input features to gate angles
- Optional extra parameters $\alpha_{q,l}$ (e.g., class-dependent coefficients)
- All these form a high-dimensional parameter vector over which a cost function is minimized.

Number of Parameters: $(3+d)N$ for a single qubit

Per layer: 3 rotation angles+feature weights (+class weights)

MINIMIZATION METHODS

HYBRID ALGORITHM

Quantum: Run the circuit & measure the outcomes

Classical: Update the parameters via classical optimizer

OPTIMIZATION CHALLENGE

The landscape being Non-convex & rugged makes local optimizers easily trapped in sub-optimal local minimum, similar to neural networks.

SGD struggles with small, non-convex sets

L-BFGS-B (Treated as Black Box) (Limited-memory
Broyden-Fletcher-Goldfarb-Shanno)

- **Efficiency:** Converges in fewer iterations compared to Gradient Descent (GD)
- **Robustness:** Highly effective even on small datasets
- **Approximation:** Builds approximation of Hessian without computing full Hessian
- **Memory:** Stores only a small number of previous updates

CLASSIFICATION RESULTS FROM PAPER

Conclusion:

A single qubit classifier can handle multi-class, multi-dimensional data and a Multi-qubit classifier can improve accuracy and reduce layers.

Parameters & Logic

$L = \text{range}(1, 10)$

Learning: $\{\theta_i\}$, $\{w_i\}$, $\{\alpha_i\}$ (when using weighted fidelity)

Using two Cost functions: Fidelity & Weighted Fidelity

Open Source Code

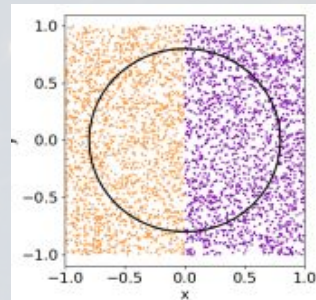
Available on GitHub:

https://github.com/AdrianPerezSalinas/universal_classifier

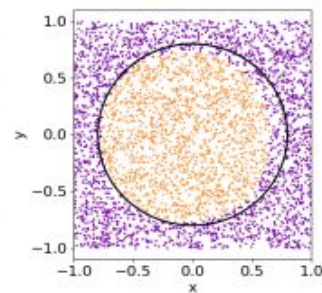


CIRCLE CLASSIFICATION

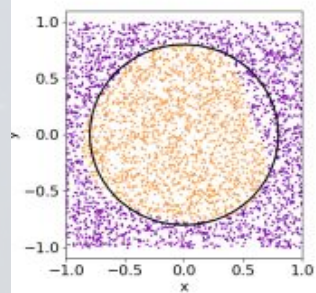
Qubits Layers	χ_f^2			χ_{wf}^2				
	1	2		1	2		4	
		No Ent.	Ent.		No Ent.	Ent.	No Ent.	Ent.
1	0.50	0.75	—	0.50	0.76	—	0.76	—
2	0.85	0.80	0.73	0.94	0.96	0.96	0.96	0.96
3	0.85	0.81	0.93	0.94	0.97	0.95	0.97	0.96
4	0.90	0.87	0.87	0.94	0.97	0.96	0.97	0.96
5	0.89	0.90	0.93	0.96	0.96	0.96	0.96	0.96
6	0.92	0.92	0.90	0.95	0.96	0.96	0.96	0.96
8	0.93	0.93	0.96	0.97	0.95	0.97	0.95	0.96
10	0.95	0.94	0.96	0.96	0.96	0.96	0.96	0.97



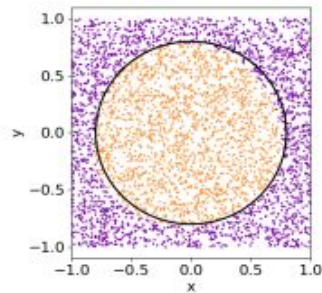
(a) 1 layer



(b) 2 layers



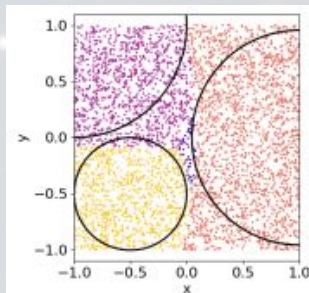
(c) 4 layers



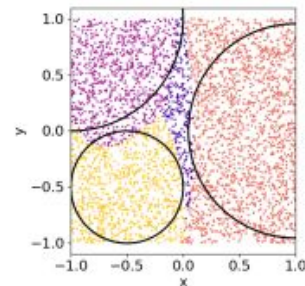
(d) 8 layers

3 CIRCLES (multiple patterns , 4 classes)

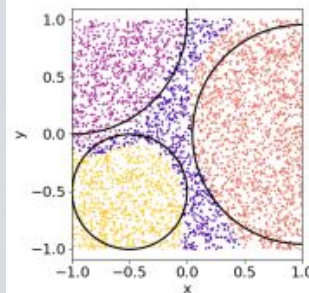
Qubits Layers	χ_f^2			χ_{wf}^2				
	1	2		1	2		4	
		No Ent.	Ent.		No Ent.	Ent.	No Ent.	Ent.
1	0.73	0.56	—	0.75	0.81	—	0.88	—
2	0.79	0.77	0.78	0.76	0.90	0.83	0.90	0.89
3	0.79	0.76	0.75	0.78	0.88	0.89	0.90	0.89
4	0.84	0.80	0.80	0.86	0.84	0.91	0.90	0.90
5	0.87	0.84	0.81	0.88	0.87	0.89	0.88	0.92
6	0.90	0.88	0.86	0.85	0.88	0.89	0.89	0.90
8	0.89	0.85	0.89	0.89	0.91	0.90	0.88	0.91
10	0.91	0.86	0.90	0.92	0.90	0.91	0.87	0.91



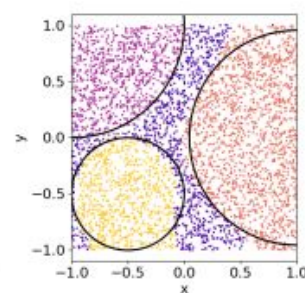
(a) 1 layer



(b) 3 layers



(c) 4 layers



(d) 10 layers

4D SPHERE (HIGHER DIMENSIONAL DATA)

4D Input Formula

$$X_1 + X_2 + X_3 + X_4 < 2/\pi$$

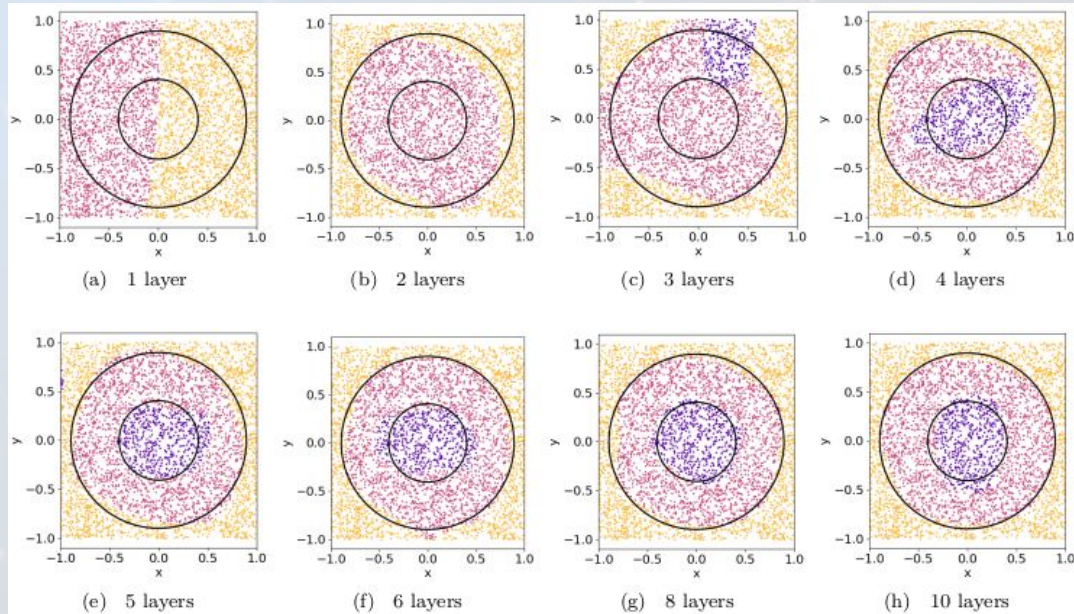
A single qubit has **3 degrees of freedom**. In higher dimensions, we split the vector and upload subsets across layers.

With enough gates, we can upload **any dimensional data**.

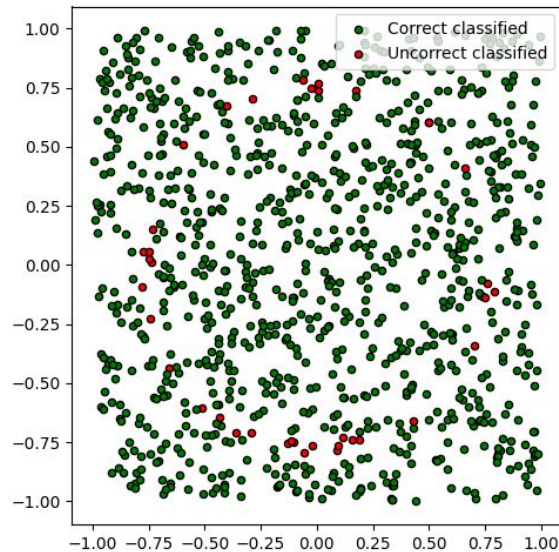
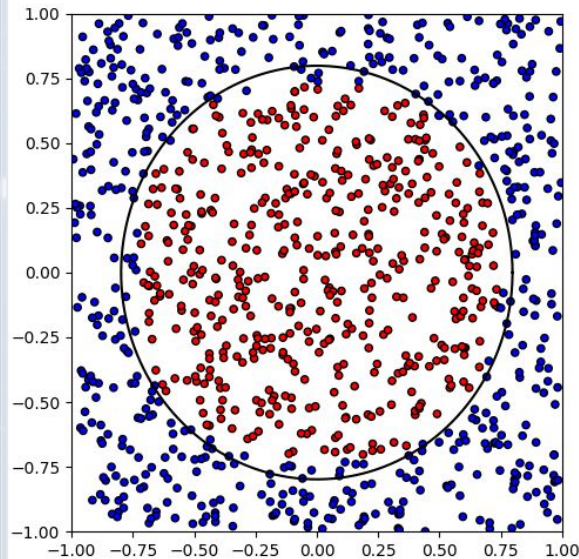
Qubits Layers	χ_f^2			χ_{wf}^2				
	1	2		1	2		4	
		No Ent.	Ent.		No Ent.	Ent.	No Ent.	Ent.
1	0.87	0.87	—	0.87	0.87	—	0.90	—
2	0.87	0.87	0.87	0.87	0.92	0.91	0.90	0.98
3	0.87	0.87	0.87	0.89	0.89	0.97	—	—
4	0.89	0.87	0.87	0.90	0.93	0.97	—	—
5	0.89	0.87	0.87	0.90	0.93	0.98	—	—
6	0.90	0.87	0.87	0.95	0.93	0.97	—	—
8	0.91	0.87	0.87	0.97	0.94	0.97	—	—
10	0.90	0.87	0.87	0.96	0.96	0.97	—	—

ANNULUS (NON-CONVEX, multi-class)

Target Problem
2D plane with 3 classes



QISKIT ADAPTATION



```
def circuit(n_qubits, n_layers, theta, w, x):  
  
    qc = QuantumCircuit(n_qubits)  
  
    for j in range(n_qubits):  
        for i in range(n_layers):  
            phi = theta[j][i] + w[j][i]*x  
  
            qc.rz(phi[2], j)  
            qc.ry(phi[0], j)  
            qc.rz(phi[1], j)  
  
    return qc
```

The background is a dark blue-grey gradient. It is decorated with a complex network of thin, glowing lines in light blue, white, and purple. These lines connect small, circular nodes of varying sizes and colors (blue, white, yellow, purple). The overall effect is that of a digital or neural network, with some areas appearing more densely connected than others.

THANK YOU

END OF PRESENTATION